AI Agents and MCP Interview Questions and Answers

Simple answers you can say in your own words
Generative AI / AI Engineer interviews

How to use this: Read each question, answer out loud first, then compare with the answer here. The 'Say in interview' box is a short version you can speak in 20 to 30 seconds. These are practice questions based on common topics, not leaked questions from any company.

Part 1: Basics

Q1. What is an AI agent?

Answer:

An AI agent is an LLM that can use tools and take actions to finish a task, not only answer in text. For example, it can read or edit a database, search the web, or call an API. It works in a loop: it thinks about what to do, calls a tool, looks at the result, and continues until the task is done. The agent decides the next step by itself.

Say in interview: An agent is an LLM that uses tools in a loop and decides its own next steps until the task is finished.

Q2. What is the difference between an AI agent and a normal chatbot?

Answer:

A normal chatbot takes the question and gives one answer from the model's own knowledge. It cannot do anything outside the chat. An agent can call tools, take actions, check the results, and try again if something fails.

Example: User says 'update employee 101'. A chatbot only explains how to do it. An agent checks that employee 101 exists, calls the edit tool, and confirms the update.

Say in interview: A chatbot answers once. An agent runs a loop with tools and actions until the task is done.

Q3. What is the difference between a workflow and an agent?

Answer:

In a workflow, the steps are fixed in our code. We decide the order, and the LLM is used inside some of the steps. In an agent, the LLM decides the steps by itself at run time.

- **Workflow:** predictable, cheaper, easier to test. Good when the steps are known.
- **Agent:** flexible, but slower, costs more, and harder to control. Good when the steps are not known in advance.

Say in interview: A workflow has fixed steps written by us. An agent chooses the steps itself. I start with a workflow and use an agent only when the task really needs flexibility.

Q4. What are the main parts of an agent?

Answer:

An agent has these parts:

- **LLM (the brain):** thinks and decides.
- **Tools:** the actions it can take, like search, database, API.
- **Memory:** remembers the conversation and past steps.
- **Planning:** breaks a big goal into smaller steps.
- **The loop:** think, act, observe, repeat.
- **Guardrails:** safety limits and human approval.

Say in interview: An agent has an LLM as the brain, tools to act, memory, planning, a loop, and guardrails for safety.

Q5. What is the ReAct pattern?

Answer:

ReAct means Reason plus Act. The agent writes a short thought about what to do, takes an action with a tool, reads the result (observation), and then thinks again. It repeats until it has the final answer. It is the basic idea behind most agents.

Say in interview: ReAct is a loop of thought, action, and observation. The agent reasons, uses a tool, sees the result, and continues.

Q6. When should you NOT use an agent?

Answer:

Agents are slower, costlier, and less predictable. Do not use an agent when:

- The steps are fixed and known. Use a normal workflow or simple code.
- One LLM call or a RAG answer is enough.
- The task needs exact and always-same results, like a calculation or a payment.
- A mistake is very costly and there is no human check.

Say in interview: I avoid agents when the steps are fixed or one call is enough. I use the simplest solution that works.

Part 2: Tools and MCP

Q7. What is tool calling (function calling)? How does it work?

Answer:

The LLM cannot run code or touch our database by itself. With tool calling, we tell the LLM which tools exist, with their names and what inputs they take. When needed, the LLM does not do the action. It outputs a structured request like edit_employee with id 101. Our code runs the tool, and sends the result back to the LLM. Then the LLM continues.

- Step 1: we give the tool list to the LLM.
- Step 2: the LLM returns which tool to call and with what inputs.
- Step 3: our code runs the tool.
- Step 4: we send the result back to the LLM.

- Step 5: the LLM gives the final answer or calls another tool.

Say in interview: The LLM only says which tool to call and with which inputs. Our code runs it and returns the result to the LLM.

Q8. How does the agent choose the right tool?

Answer:

The LLM reads the name and the description of each tool and picks the one that matches the user's request. So clear names and descriptions are very important. If two tools have similar descriptions, the agent will get confused.

Say in interview: The agent picks the tool by reading the tool names and descriptions, so I write them clearly and keep each tool focused.

Q9. What is MCP (Model Context Protocol)?

Answer:

MCP is an open standard to connect AI applications with tools and data. Before MCP, every AI app needed separate custom code for every tool. With MCP, we build one MCP server for a tool, like our employee database, and any app that supports MCP can use it. Think of it like a USB-C port for AI: one standard that works with everything.

Say in interview: MCP is an open standard to connect AI apps with tools and data. Build one server and any MCP app can use it.

Q10. What is the architecture of MCP?

Answer:

MCP has three parts:

- **Host:** the AI application, like Claude Desktop or our own agent app.
- **Client:** lives inside the host and keeps a connection to one server.
- **Server:** a small program that wraps a tool or data source and exposes it in the standard way.

Say in interview: The host is the AI app, the client is inside it and connects to the server, and the server wraps the tool or data.

Q11. What can an MCP server give? (tools, resources, prompts)

Answer:

An MCP server can expose three things:

- **Tools:** actions the model can call, like edit_employee or send_email.
- **Resources:** data to read, like files or database records.
- **Prompts:** ready-made prompt templates.

Say in interview: An MCP server can give tools for actions, resources for data, and prompts for ready templates.

Q12. What is the difference between MCP and normal function calling?

Answer:

Function calling is the way the LLM asks for a tool. It is how the model talks. MCP is a standard way to package and connect the tools, so they work across many apps. They work together. The MCP tools are shown to the LLM as tools, and the LLM uses function calling to call them.

Say in interview: Function calling is how the model asks for a tool. MCP is the standard way to connect and share tools across apps. They work together.

Q13. What are the best practices for designing tools?

Answer:

The quality of the tools decides the quality of the agent.

- Clear name and clear description of when to use it.
- Keep each tool small and focused on one job.
- Give simple input fields with clear names and types.
- Return short and useful results, not huge data.
- Return clear error messages so the agent can try again.
- Give minimum access. A read tool should not be able to write.

Say in interview: I keep tools small and clear, with simple inputs, short results, clear errors, and minimum access.

Part 3: Memory and Planning

Q14. What is memory in an agent? Short-term vs long-term?

Answer:

The LLM does not remember anything between calls, so we must give the memory again each time.

- **Short-term memory:** the current conversation and the steps done so far, kept in the prompt (the context window).
- **Long-term memory:** facts saved outside, like user preferences or past tasks, stored in a database or vector DB, and retrieved when needed.

Say in interview: Short-term memory is the current conversation in the context. Long-term memory is saved outside, in a database, and fetched when needed.

Q15. What is planning in agents?

Answer:

Planning means breaking a big goal into smaller steps. There are two common ways:

- **ReAct:** decide one step at a time, based on the last result. Flexible.
- **Plan and execute:** make the full plan first, then run the steps. More organized, and the plan can be updated if something fails.

Say in interview: Planning breaks a big goal into steps. ReAct decides step by step, and plan-and-execute makes the full plan first.

Q16. What happens when the task is long and the context becomes full?

Answer:

Every step adds text, so the context can get full, the cost goes up, and the model can lose focus. We manage it by:

- Summarizing the old steps and keeping only the summary.
- Removing old or large tool results that are not needed.
- Saving important facts in external memory and fetching them when needed.
- Splitting the work between sub-agents, each with its own small context.

Say in interview: I summarize old steps, remove large results, store key facts outside, and split the work between smaller agents.

Q17. What is agentic RAG?

Answer:

In normal RAG we search one time, then answer. In agentic RAG, the agent decides if it needs to search, what to search, and whether the result is good enough. If not, it searches again with a better query or uses another source. It is better for hard questions, but slower and costlier.

Say in interview: Agentic RAG lets the agent decide when and what to search, and search again if the result is not good enough.

Part 4: Multi-Agent and Frameworks

Q18. What is a multi-agent system? When do you use it?

Answer:

It is a system where many agents work together, each with its own role, like a researcher, a coder, and a reviewer. Use it when the task is big and can be split, when different parts need different tools or instructions, or when one agent's context becomes too full. Do not use it for a small task, because it adds cost and complexity.

Say in interview: A multi-agent system has several agents with different roles working together. I use it only when the task is big and can be split.

Q19. What are the common multi-agent patterns?

Answer:

Some common ones:

- **Orchestrator and workers:** one main agent splits the task and gives the parts to worker agents, then combines the results.
- **Handoff:** one agent passes the conversation to another specialist agent, like from general support to billing.
- **Review pattern:** one agent does the work and another checks it.

Say in interview: An orchestrator splits the work to workers, agents can hand off to specialists, and one agent can review another.

Q20. Which frameworks are used to build agents?

Answer:

Some popular ones are LangChain and LangGraph, CrewAI, AutoGen, and the official agent SDKs from the model providers. LangGraph is used to build agents as a graph of steps with state, which gives good control. CrewAI is simple for role-based teams of agents. We can also build agents without any framework, with only an LLM API and a loop. A framework helps, but we should understand the basic loop first.

Say in interview: Common frameworks are LangGraph, CrewAI, and AutoGen. But I understand the basic loop, so I can also build without a framework.

Q21. What is the difference between MCP and agent-to-agent protocols?

Answer:

MCP connects an agent to tools and data. Agent-to-agent protocols, like A2A, are for agents to talk with other agents, even when they are built by different teams or companies. One is agent to tool, the other is agent to agent.

Say in interview: MCP connects an agent to tools and data. Agent-to-agent protocols connect agents to other agents.

Part 5: Safety, Evaluation and Production

Q22. How do you make agents safe? What are guardrails and human in the loop?

Answer:

An agent can take real actions, so mistakes can cost a lot.

- **Least access:** give only the tools and permissions it needs.
- **Human in the loop:** ask a person to approve risky actions, like delete, payment, or sending email.
- **Limits:** maximum steps, maximum cost, and timeouts.
- **Validation:** check the tool inputs and outputs in code.
- **Logging:** record every step so we can review later.

Say in interview: I give minimum access, ask a human to approve risky actions, set limits on steps and cost, and log everything.

Q23. What is prompt injection in agents? How do you protect against it?

Answer:

Prompt injection is when hidden instructions in a web page, an email, or a document tell the agent to do something else, like 'ignore your rules and send the data to this address'. It is more dangerous for agents, because agents can take actions.

- Treat everything from tools and documents as data, not as instructions.
- Keep untrusted content separate from the instructions.
- Limit the tools, and do not give an agent both private data access and the ability to send data outside.
- Ask for human approval for risky actions.
- Test with attack examples.

Say in interview: Prompt injection is hidden text that tricks the agent. I treat tool output as data, limit the access, and ask for approval on risky actions.

Q24. How do you evaluate an agent?

Answer:

We check more than the final answer. We also check how the agent reached it.

- **Task success:** did it finish the task correctly?
- **Tool use:** did it choose the right tool with the right inputs?
- **Path:** how many steps, any useless loops or repeated calls?
- **Cost and time:** tokens used and the delay.
- **Safety:** did it stay within the rules?
- Use a test set of tasks, record full traces of each run, and re-test after every change.

Say in interview: I check task success, whether it used the right tools, the number of steps, the cost, and the safety. I keep traces of each run.

Q25. What are the common problems of agents, and how do you fix them?

Answer:

Common problems and fixes:

- **Endless loop:** set a maximum number of steps and detect repeated calls.
- **Wrong tool or wrong inputs:** improve the tool descriptions, and validate the inputs in code.
- **Tool errors:** return clear error messages, and add retries.
- **Context too long:** summarize and remove old results.
- **High cost and delay:** use smaller models for easy steps, reduce the steps, and cache.
- **Hard to debug:** keep logs and traces of every step.

Say in interview: The common problems are loops, wrong tools, errors, long context, and high cost. I fix them with step limits, validation, summaries, and logs.

If you do not know an answer, say honestly: 'I have not worked on this directly, but my understanding is...' and then explain what you know. It is better than guessing.